Graph Embeddings and GNN-Based Community Detection

in Social and Computer Networks

Nima Kamali Lassem · Nicola Mambelli

February 2026 · CentraleSupélec / Paris-Saclay

Knowledge Graph Fundamentals

Core Concepts

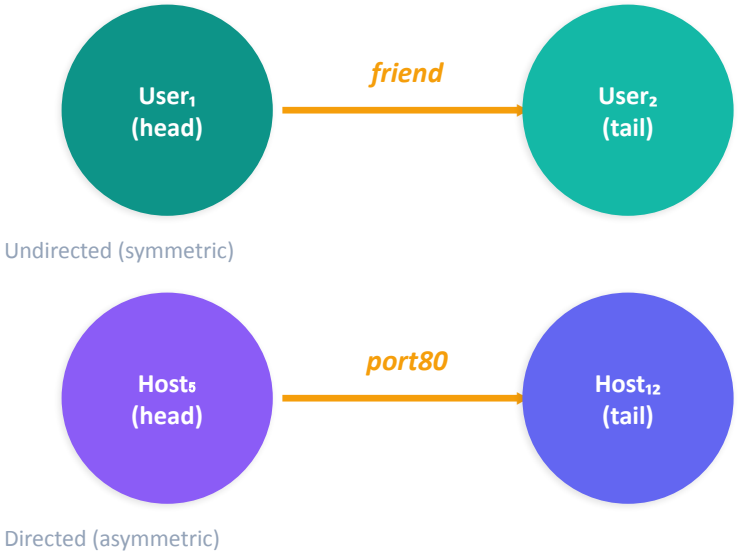
Nodes (Entities)
Represent real-world objects: users, hosts, devices. Each node can have attributes (features).

Edges (Relationships)
Connections between nodes. Can be directed (client→server) or undirected (friendship). Can carry types and weights.

Triples (h, r, t)
The fundamental unit of a knowledge graph: (head, relation, tail). E.g., ($User_1$, friend, $User_2$) or ($Host_5$, port80, $Host_{12}$).

Community Structure
Groups of densely connected nodes. Reflects functional roles, social groups, or organizational structure.

Triple Representation: (h, r, t)



Two Contrasting Datasets

LastFM Asia Social Network

7,624 nodes
(music users from 18 Asian countries)

27,806 undirected edges
(friendships — 1 relation type, 100% symmetric)

Rich node features
(7,842 binary artist preferences ⟶ 128-dim SVD)

Density: 0.096% · Sparse

Ground truth: 18 country labels
Class imbalance: 98:1 ratio (largest vs smallest)

Description: Social network of Last.fm users from 18 Asian countries, with friendships as edges and music listening preferences as node attributes.

Cisco g21 Network Traffic

52 nodes
(network hosts with functional roles)

Directed edges, 2,157 relation types
(port/protocol combos — 94.2% asymmetric)

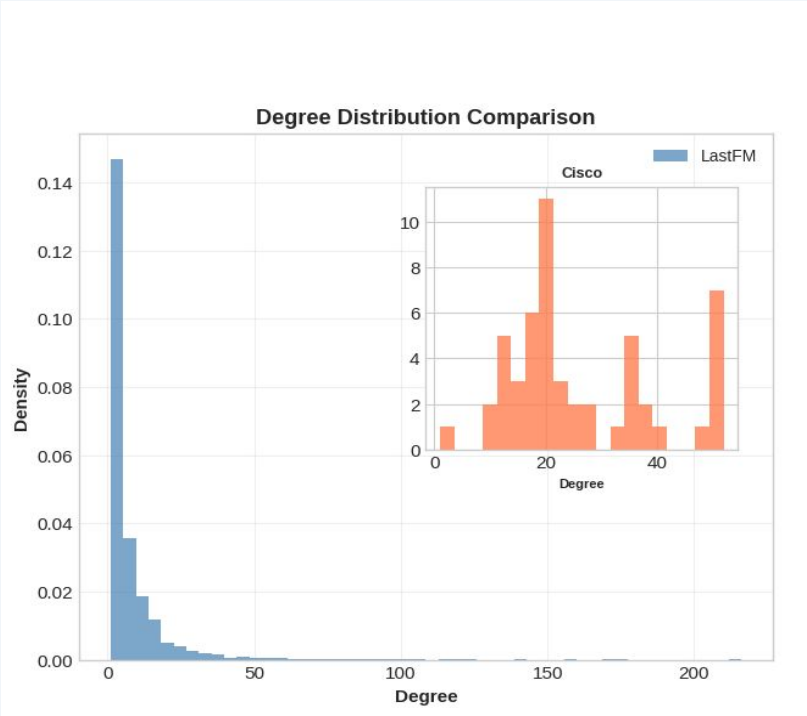
No native node features
(engineered from port usage patterns)

Density: 96% raw ⟶ 51% filtered

Ground truth: 23 functional groups
(verified by actual host function)

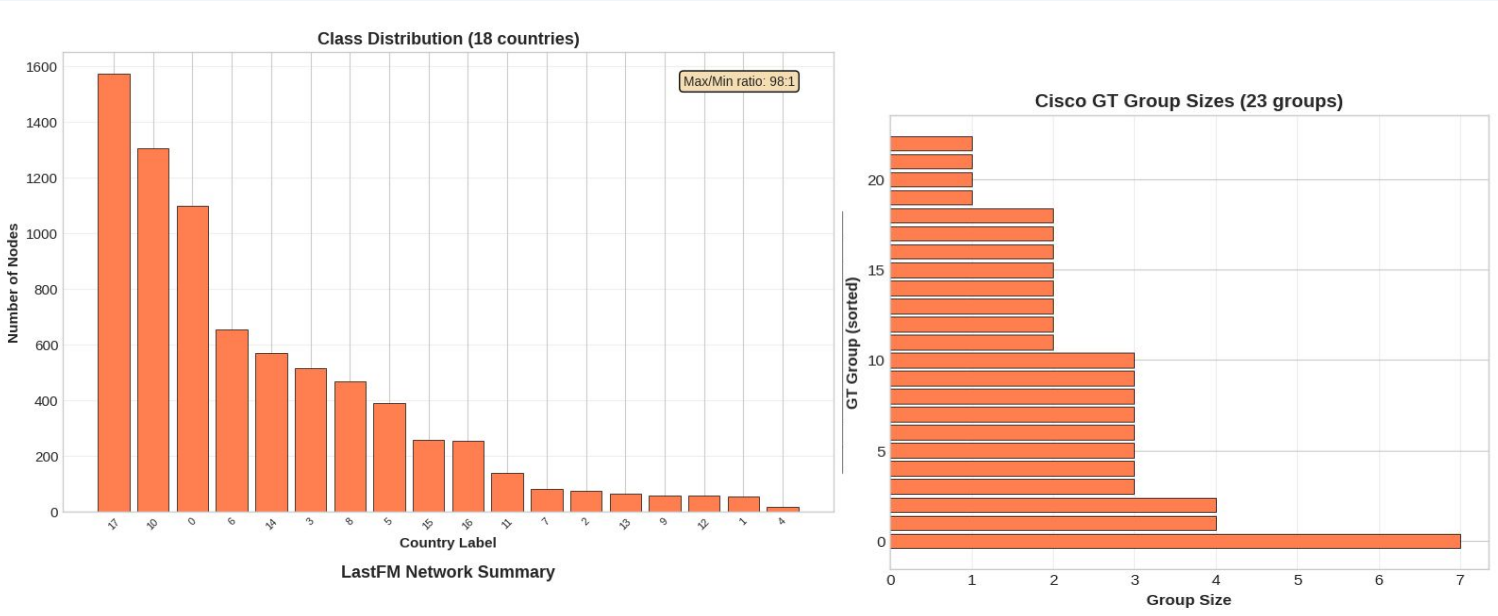
Description: Network of 52 hosts representing communication traffic between devices, where edges capture port/protocol interactions reflecting functional roles.

Dataset Characterization: Properties That Determine Model Choice



Why this matters: Symmetry, density, and relation count directly predict which models will succeed or fail.

Dataset Characterization: Properties That Determine Model Choice



Why this matters: Symmetry, density, and relation count directly predict which models will succeed or fail.

Summary of Previous Work

Part 1: Classical Community Detection · Part 2 (early): Spectral Clustering & Node2Vec

Brief recap — these methods set the baseline for the main results

Part 1: Classical Community Detection — Baselines

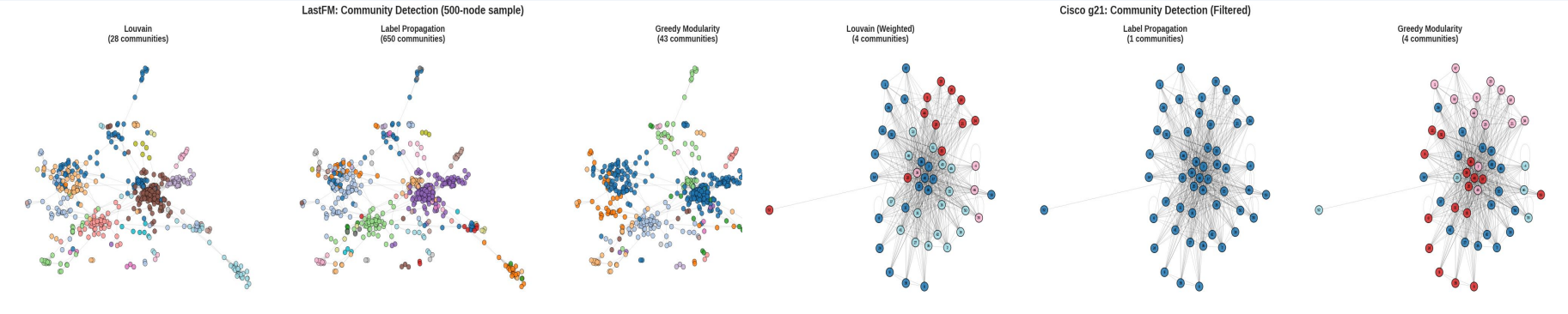
LastFM (Undirected, No Ground Truth)

Algorithm	Modularity	Conductance	Communities
Louvain ✓	**0.815**	0.135	27
Label Propagation	0.752	0.435	650
Greedy Modularity	0.796	0.163	43

Cisco g21 (Directed, With Ground Truth)

Algorithm	Jaccard	NMI	Communities
Louvain	**0.531**	0.165	2
Girvan-Newman	0.322	**0.296**	**9**
Label Propagation	0.077	0.000	1

Key insight: Louvain is our strongest baseline. Dense networks (Cisco) cause Label Propagation and Infomap to collapse into a single community.



Spectral Clustering: Eigenvalues of the Graph Laplacian

Pipeline

Adjacency Matrix A

↓

Normalized Laplacian L_sym

↓

k smallest eigenvectors

↓

K-Means on eigenvectors

↓

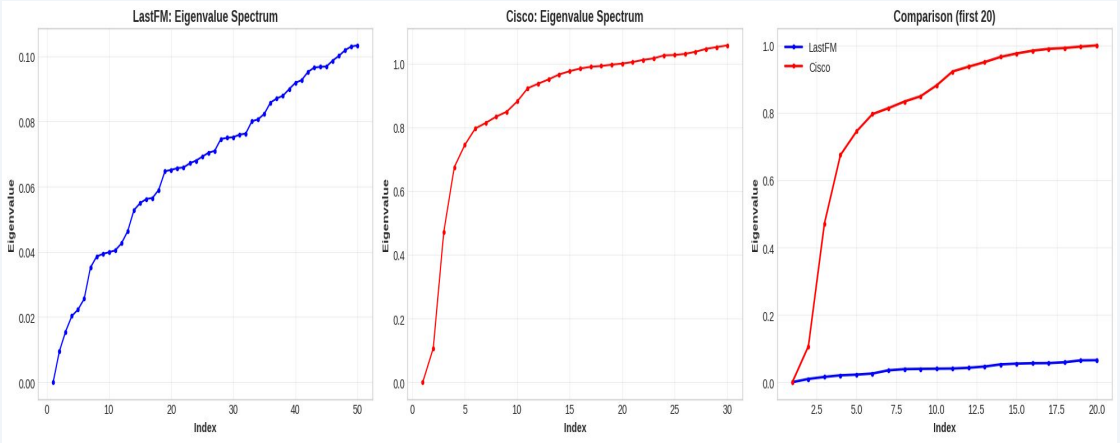
Community Assignments

L_sym = D^(-1/2) L D^(-1/2)
Eigengap heuristic selects k

Results

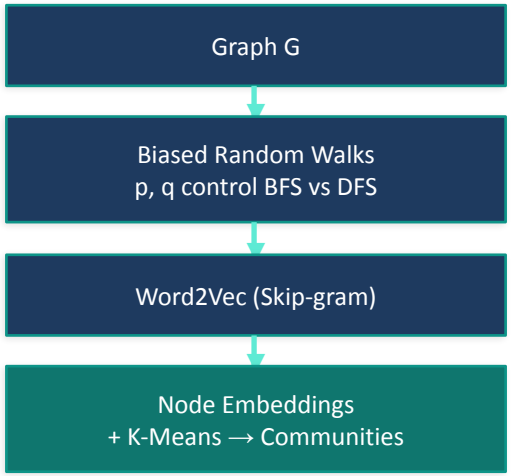
Dataset	k	Mod.	NMI	ARI
LastFM	18	0.799	**0.659**	**0.671**
Cisco	23	0.205	**0.863**	0.496

Cisco achieves NMI=0.863 despite low modularity — functional groups aren't dense subgraphs.



Node2Vec: Biased Random Walks → Shallow Embeddings

Pipeline



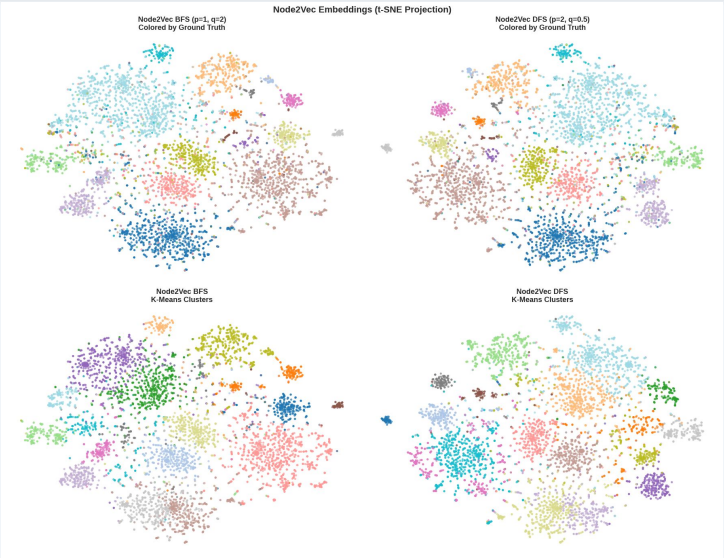
BFS (p=1, q=2): local structure
DFS (p=2, q=0.5): global homophily
num_walk LastFM: 10 (Original Paper)
num_walk Cisco: 20 (more sparse) (low for W2V)

t-SNE stands for **t**-distributed Stochastic Neighbor Embedding, a nonlinear dimensionality reduction method mainly used for visualization, not for downstream modeling.

LastFM Results

Config	Mod.	NMI	ARI
BFS (p=1, q=2)	0.778	0.623	0.533
DFS (p=2, q=0.5)	**0.804**	**0.635**	**0.561**

DFS > BFS: global structure captures more community signal. Neither beats Louvain on modularity or spectral on NMI. Cisco: poor (too dense for random walks).



Main Contributions

GCN · Link Prediction · Knowledge Graph Embeddings · Transfer Learning

The core of our work: deep learning and knowledge graph embedding methods

GCN for Node Classification: Theory & Pipeline

Why GCN?

Only method that uses both graph structure AND node features

Message passing: each node aggregates neighbor information

$$H' = \hat{A} \cdot H \cdot W$$

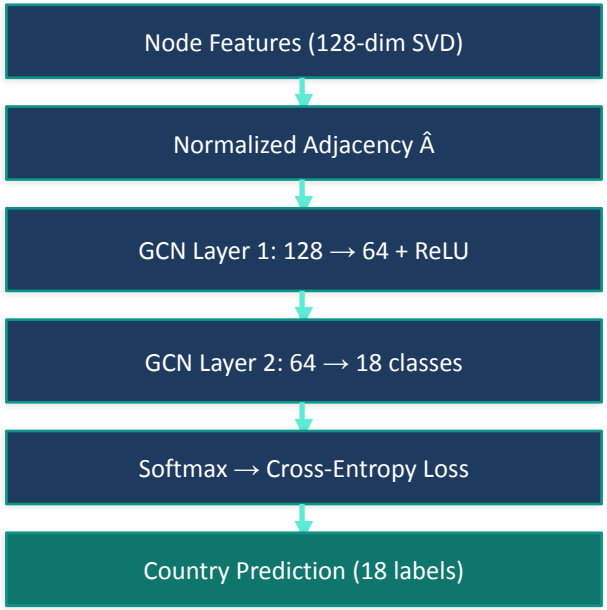
where $\hat{A} = \tilde{D}^{(-1/2)} \tilde{A} \tilde{D}^{(-1/2)}$ (normalized adjacency)

Architecture: 2 GCN layers (128→64→18 classes)
Trained with cross-entropy on 70% labeled nodes

What makes GCN different from previous methods?

Method	Structure	Features	Learned
Spectral	✓	✗	✗
Node2Vec	✓	✗	✓
GCN	✓	✓	✓

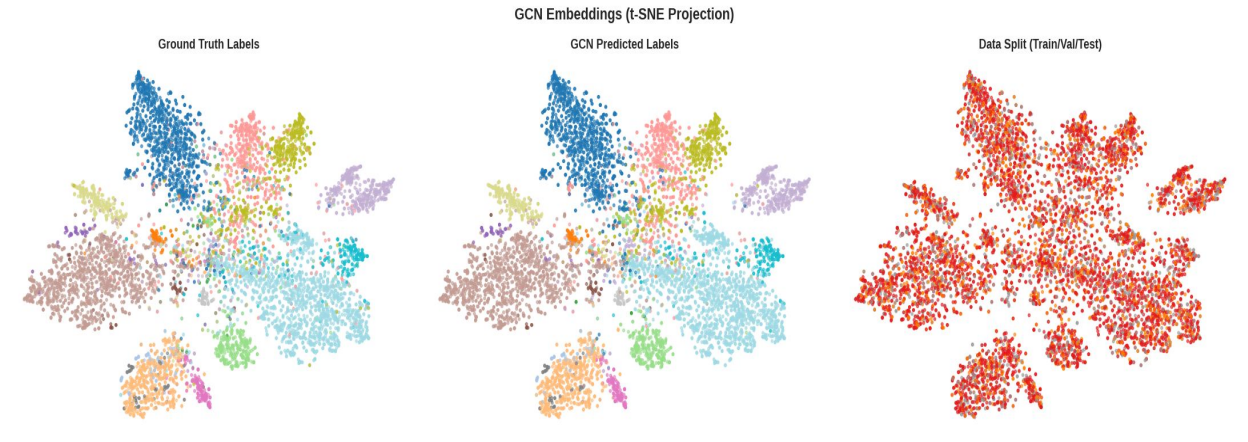
Node Classification Pipeline(LastFM)



GCN Results: Training & Embeddings

LastFM Test Accuracy: **89.4%**

Cisco Test Accuracy: **20.0%**



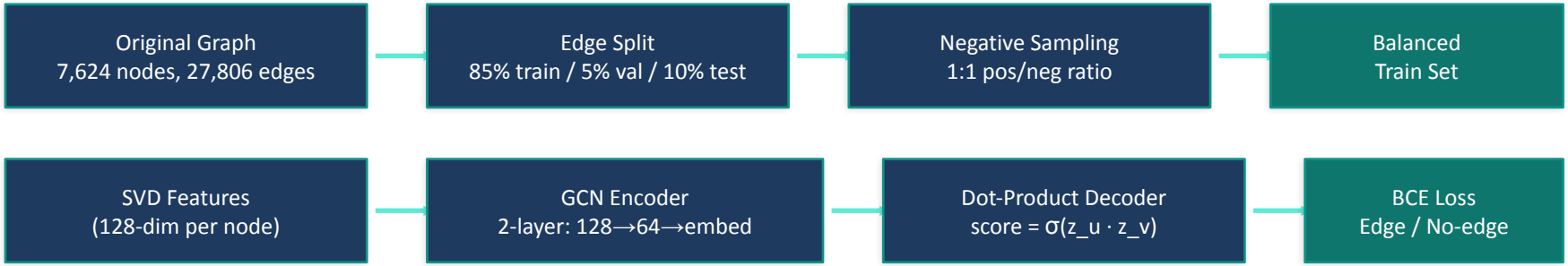
Cisco: Port-based features

Binary vector of port usage patterns — web servers on 80/443, DNS on 53, etc.

Only 52 nodes → higher variance, smaller hidden dim (32 vs 64)

20% accuracy with high variance reflects extreme difficulty: 23 classes, ~2 nodes/class

Link Prediction with GNN: Pipeline (LastFM)



Two Evaluation Paradigms:

Binary Edge Classification
AUC, Average Precision

"Does an edge exist between u and v?"
Binary yes/no → GCN trained for this (BCE)
AUC = 0.944 (Area Under the ROC Curve) ·
AP(Average Precision) = 0.948

Entity Ranking (head + tail)
MR, MRR, Hits@K

"Rank the correct target among ALL 7,624 nodes"
GCN NOT trained for this → much worse performance
Hits@10 = 12.82% · MRR = 0.058

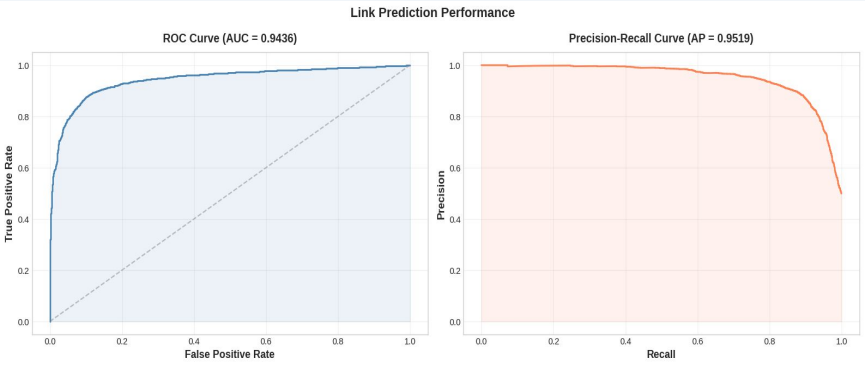
Key Insight: Why Loss Functions Determine What Embeddings Learn

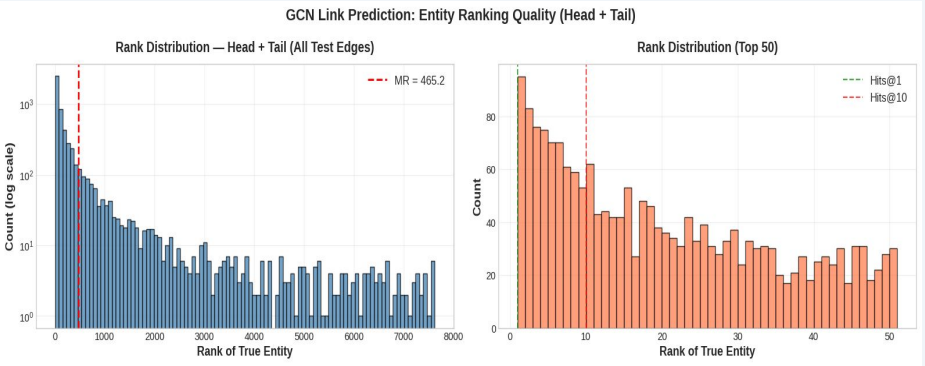
Binary Cross-Entropy (GCN) vs Margin-Based Ranking (KGE models)

BCE asks: "Is this pair an edge?" — just needs scores above/below 0.5. Easy: most random pairs are clearly not edges.

Ranking loss asks: "Can you rank the correct target ABOVE all other candidates?" — requires fine-grained score differences among plausible neighbors.

Result: GCN achieves AUC = 0.944 (great at discrimination) but only Hits@10 = 12.82% (poor at ranking).
TransE/RotatE trained with ranking loss → directly optimized for the harder ranking task.





Knowledge Graph Embedding Models

TransE · RotatE · DistMult — Dedicated link prediction models with ranking-based loss

TransE: Translational Embeddings

Scoring Function

$$f(h, t) = ||h + r - t||_2$$

Relation r acts as a translation from head to tail.
Low score = triple is likely true.

Loss: MarginRankingLoss (γ=1.0)
Push correct triples' scores below corrupted triples by at least margin γ. All negatives weighted equally.

Known Limitations (from lecture)

Symmetry: Forces $r \longrightarrow 0$, collapsing h and t to same point
1-to-N: Forces all tails to collapse ($t_1 = t_2$)
N-to-N: Same collapse in both directions
✓ **Composition,** ✓ **Antisymmetry**

Impact on Our Datasets

LastFM: Expect FAILURE

- 100% symmetric edges ⟶ $r \approx 0$
- 77% nodes have multiple friends ⟶ 1-to-N collapse
- Single relation type ⟶ zero relational capacity
- High clustering ⟶ cascading embedding collapse

Cisco: Expect SUCCESS

- 94% asymmetric ⟶ no symmetry problem
- 2,157 relation types ⟶ full multi-relational power
- Simple geometry generalizes with sparse data (~2.3 triples/relation)

Python library: PyKEEN

RotatE: Rotational Embeddings in Complex Space

Scoring Function

$$t = h \circ r, \quad |r_i| = 1 \text{ (unit complex numbers)}$$

$$f(h, t) = ||h \circ r - t||$$

Each $r_i = e^{(i\theta_i)}$ — a rotation angle in complex plane.
Relations as rotations, not translations.

Loss: NSSALoss (self-adversarial, γ=6.0, α=1.0)
Weights harder negatives more → focuses training on most informative examples.

Capabilities (solves TransE's limitations)

- ✓ **Symmetry:** Rotation by π ($r_i = -1$), applying twice returns to start
- ✓ **1-to-N:** Different complex embeddings map correctly under same rotation
- ✓ **Composition:** $r_3 = r_1 \circ r_2$ (rotation composition)
- ✓ **Inversion:** $r_2 = \bar{r}_1$ (conjugate = inverse rotation)

NSSA Loss: Self-Adversarial

Standard negative sampling treats all corrupted triples equally — but some negatives are easy (clearly wrong) and some are hard (plausible).

NSSA uses the model's own scores to weight negatives: harder negatives get higher weight.

Why RotatE + NSSA?

RotatE's rich complex-space embeddings quickly learn easy negatives. NSSA keeps training challenging by focusing on the negatives the model is most confused about.

Why TransE + MarginRanking?

TransE is simpler — uniform negative sampling gives stable, predictable gradients. Its limited capacity benefits from diverse negatives.

Python library: PyKEEN

DistMult & Three-Model Comparison

DistMult: Bilinear Model

$$f(h,t) = h^T \text{diag}(r)\, t = \Sigma_i\, h_i \cdot r_i \cdot t_i$$

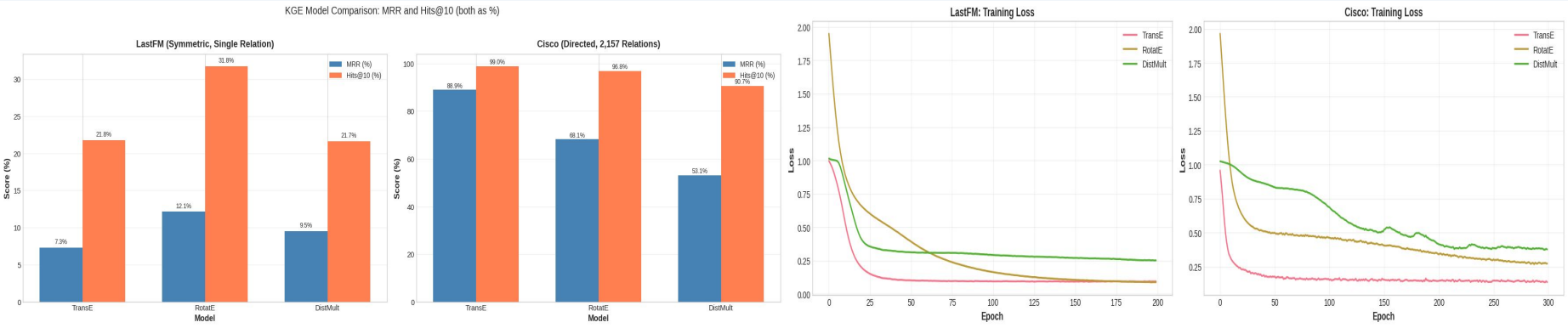
Inherently symmetric: f(h,r,t) = f(t,r,h) always

Natural fit for symmetric graphs (LastFM)
Fatal flaw for directed graphs (Cisco)

Model Architecture Comparison

Model	Scoring	Symmetry	Params/rel
TransE	$\|\|h+r-t\|\|$	Anti-sym	d
RotatE	$\|\|h \circ r-t\|\|$	Both	2d
DistMult	$h^T \text{diag}(r)\, t$	Always sym	d

One model breaks on symmetry (TransE), one handles everything (RotatE), one assumes symmetry (DistMult)



Link Prediction Results: LastFM (Symmetric, 1 Relation)

Metric	GCN (h+t)	TransE	RotatE	DistMult
MR	449	323	**323**	402
MRR	0.058	0.073	**0.121**	0.095
Hits@1	1.74%	**0.00%**	**2.33%**	3.40%
Hits@10	12.82%	21.77%	**31.77%**	21.66%

RotatE wins on LastFM — exactly as theory predicts:

TransE Hits@1 = 0%: Symmetric edges force $r \approx 0$ → all nodes equidistant → can NEVER rank correct target first. Compounds with single-relation bottleneck and cascading embedding collapse.

RotatE MRR 66% higher than TransE: Models symmetry as rotation by π without collapse. Cross-dimensional expressiveness matters at scale (7,624 entities).

DistMult (MRR=0.095): Built-in symmetry avoids TransE's collapse but lacks RotatE's cross-dimensional power. Hits@1=3.40% is actually highest — precise top-1 predictions but poor recall.

Link Prediction Results: Cisco (Directed, 2,157 Relations)

Metric	TransE	RotatE	DistMult
MRR	**0.848**	0.682	0.482
Hits@1	**74.5%**	48.3%	22.6%
Hits@10	**98.2%**	96.2%	90.7%

TransE dominates Cisco — opposite ranking from LastFM:

No symmetry problem: 94.2% asymmetric edges → TransE's main weakness is irrelevant.

Parameter efficiency: TransE has 138K vs RotatE's 276K relation params. With only ~2.3 examples per relation, RotatE overfits.

DistMult's fundamental flaw: Cannot distinguish (h,r,t) from (t,r,h) on 94% asymmetric edges. Scores contradictory triples identically → training signal diluted.

Simple geometry suffices: 52 entities = small ranking space. Straight-line translations are enough; RotatE's rotational complexity is wasted.

The Key Lesson: Dataset Properties Determine the Winner

Dataset Property	Favors TransE	Favors RotatE	Favors DistMult
Symmetric edges	✗ No	✓ **Yes**	✓ **Yes**
Asymmetric/directed	✓ **Yes**	— Neutral	✗ No
Many relations, sparse data	✓ **Simpler**	✗ Overfits	✗ No
Single relation, dense data	✗ No	✓ **Yes**	✓ **Yes**
Small entity space	✓ **Yes**	✗ No	✗ No
Large entity space	— Neutral	✓ **Yes**	— Neutral

LastFM → RotatE wins
Symmetric, single relation, large

Cisco → TransE wins
Directed, 2157 relations, sparse, small

This is exactly what the lecture's summary table predicts in theory. Our experiments on two real datasets confirm it in practice.

Transfer Learning: KGE → GCN

Step 1: Train KGE
TransE/RotatE/DistMult
for entity ranking

↓

Step 2: Extract
Entity Embeddings
(128/256-dim vectors)

↓

Option A:
KGE only

Option B:
Original + KGE
(concatenate)

↓

Step 3: GCN with
KGE input features

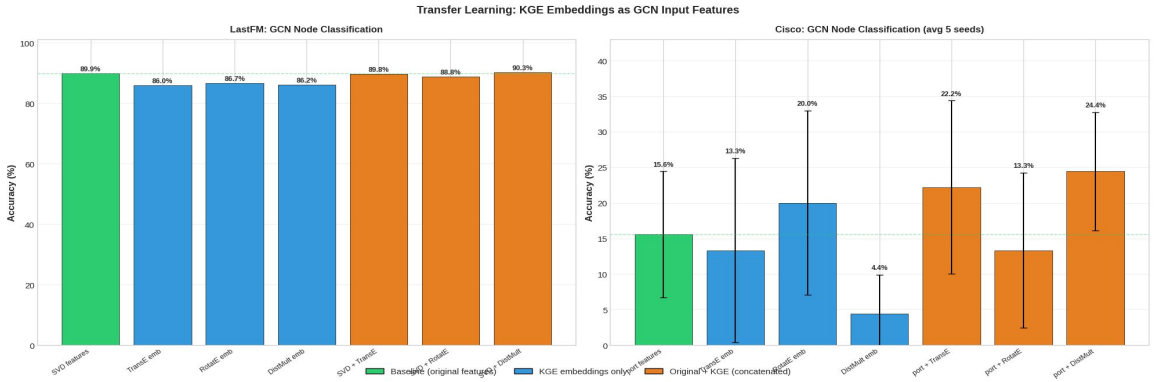
↓

Node Classification
+ Community Detection

Why transfer learning works here:

1. KGE training is unsupervised — only needs graph edges, not labels
2. KGE captures relational structure (who connects to whom via which relation)
3. GCN adds neighborhood aggregation + non-linear transformation on top
4. Concatenation combines content features (SVD/port) with structural features (KGE)

KGE encodes local relational patterns → GCN adds global neighborhood context.
This combination is more informative than either alone.



Transfer Learning Results: Classification & Community Detection

LastFM Node Classification

Variant	Test Acc
GCN (SVD)	89.4%
GCN (TransE)	85.9%
GCN (RotatE)	86.8%
GCN (DistMult)	86.0%
GCN (SVD+TransE)	89.9%
GCN (SVD+RotatE)	89.1%
GCN (SVD+DistMult)	**90.0%**

Cisco (avg over 5 seeds)

Variant	Test Acc
GCN (port)	20.0% ± 17.8%
GCN (TransE)	8.9% ± 13.0%
GCN (RotatE)	20.0% ± 20.4%
GCN (DistMult)	11.1% ± 7.0%
GCN (port+TransE)	**37.8% ± 11.3%**
GCN (port+RotatE)	22.2% ± 14.1%
GCN (port+DistMult)	11.1% ± 9.9%

Key Findings:

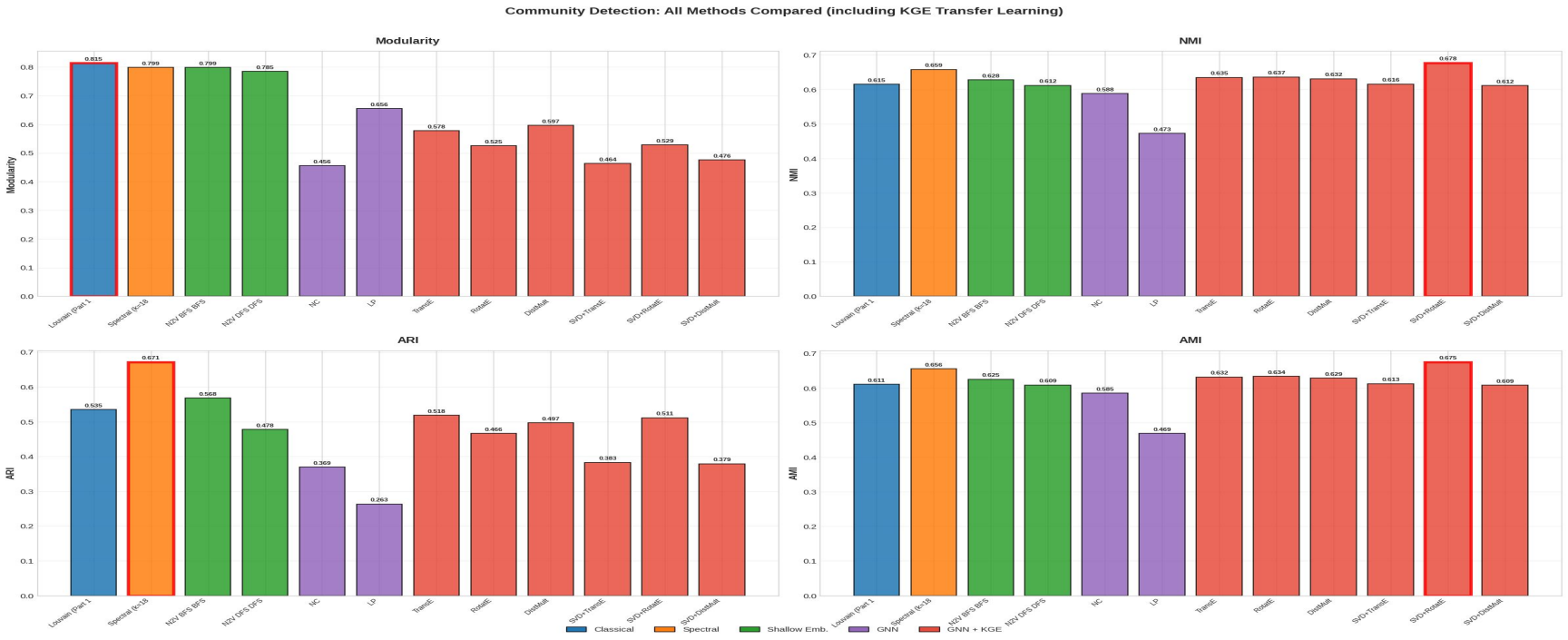
KGE-only features (85.9–86.8%): Graph structure alone carries substantial class-relevant signal — close to SVD baseline.
SVD+DistMult (90.0%) best on LastFM: DistMult's symmetric embeddings complement content features. Modest gain since SVD already captures social homophily.
Port+TransE (37.8%) best on Cisco: Nearly doubles port-only baseline. TransE's strong relational embeddings (MRR=0.848) transfer well.
Community Detection: GCN(SVD+RotatE) achieves best NMI (0.680) across ALL methods.

Comprehensive Comparison: All Methods on LastFM

Method	Type	Mod.	NMI	ARI	AMI
Louvain (Part 1)	Classical	**0.815**	0.622	0.535	0.611
Spectral (k=18)	Spectral	0.799	**0.659**	**0.671**	**0.656**
Node2Vec DFS	Shallow Emb.	0.804	0.635	0.541	0.624
GCN (Node Classif.)	GNN	0.449	0.589	0.366	0.588
GCN (Link Pred.)	GNN	0.674	0.477	0.256	0.486
GCN (SVD+RotatE)	GNN+KGE	0.713	**0.680**	0.595	**0.677**

Louvain leads on modularity (0.815). **GCN(SVD+RotatE) achieves the highest NMI (0.680)** — surpassing spectral clustering (0.659) and all other methods. Transfer learning from KGE captures community structure best.

Comprehensive Comparison: All Methods on LastFM



Key Findings

1 Modularity ≠ ground truth alignment

Louvain wins modularity but spectral better recovers true communities (NMI 0.659 vs 0.622). On Cisco, spectral achieves NMI=0.863 with low modularity.

2 AUC ≠ ranking quality

GCN gets AUC=0.944 but Hits@10=12.82%. Ranking metrics (MRR, Hits@K) reveal true link prediction quality.

3 Simpler can be better

For community detection, Louvain beats most complex methods. For Cisco link prediction, TransE (MRR=0.848) beats RotatE (0.682).

4 No universal winner

RotatE wins on symmetric data (LastFM), TransE wins on directed/sparse (Cisco), DistMult collapses on asymmetric data.

5 Loss function shapes embeddings

Margin-based ranking loss → good at entity ranking. BCE → good at binary classification. The loss matters as much as architecture.

6 KGE embeddings transfer to GCN

Concatenation improves both datasets. GCN(SVD+RotatE) achieves best community NMI (0.680) across all methods.

Method Selection Guide: Matching Models to Data Properties

Method	Best For	Weakness
Louvain	Maximizing modularity, dense subgraphs	Doesn't align well with ground truth labels
Spectral	Recovering ground truth (NMI=0.659)	Requires choosing k, no learned embeddings
Node2Vec	General-purpose embeddings (Mod=0.804)	Slightly worse than Louvain at communities
GCN	Classification (89.4%), binary LP (AUC=0.944)	Poor at ranking (Hits@10=12.82%)
GCN+KGE	**Community detection (NMI=0.680), combined**	Requires KGE pre-training step
TransE	Directed, multi-relational, sparse (MRR=0.848)	Cannot handle symmetry
RotatE	Symmetric/mixed graphs (MRR=0.121)	Overfits with sparse per-relation data
DistMult	Symmetric bilinear patterns (MRR=0.095)	Cannot model directionality at all

Cross-dataset validation is essential. Testing on only one dataset would produce misleading conclusions about model rankings.

Conclusions

1. ## GCN(SVD+RotatE) achieves best community NMI (0.680)

   Surpasses spectral clustering (0.659) and Louvain (0.622) by combining content + structural features through transfer learning from KGE.

2. ## KGE models with ranking loss outperform GCN for link prediction

   RotatE Hits@10=31.8% vs GCN 12.82% — the loss function determines what embeddings optimize for.

3. ## Dataset properties determine model rankings

   RotatE wins on symmetric LastFM, TransE wins on directed Cisco, DistMult collapses on asymmetric data. No universal winner.

4. ## Transfer learning from KGE to GCN is effective

   Unsupervised KGE embeddings provide complementary structural signal. Concatenation improves both classification and community detection.

Thank You

Questions?

github.com/NimakamaliLassem/Massive-Graph

References

TransE: Bordes et al. (2013) NeurIPS · RotatE: Sun et al. (2019) ICLR · DistMult: Yang et al. (2015) ICLR
GCN: Kipf & Welling (2017) ICLR · Node2Vec: Grover & Leskovec (2016) KDD · Spectral: Von Luxburg (2007)
LastFM Dataset: Rozemberczki & Sarkar (2020) CIKM · PyKEEN: Ali et al. (2021) JMLR
Louvain: Blondel et al. (2008) · Word2Vec: Mikolov et al. (2013) NeurIPS

Appendix

Node2Vec - Parameters

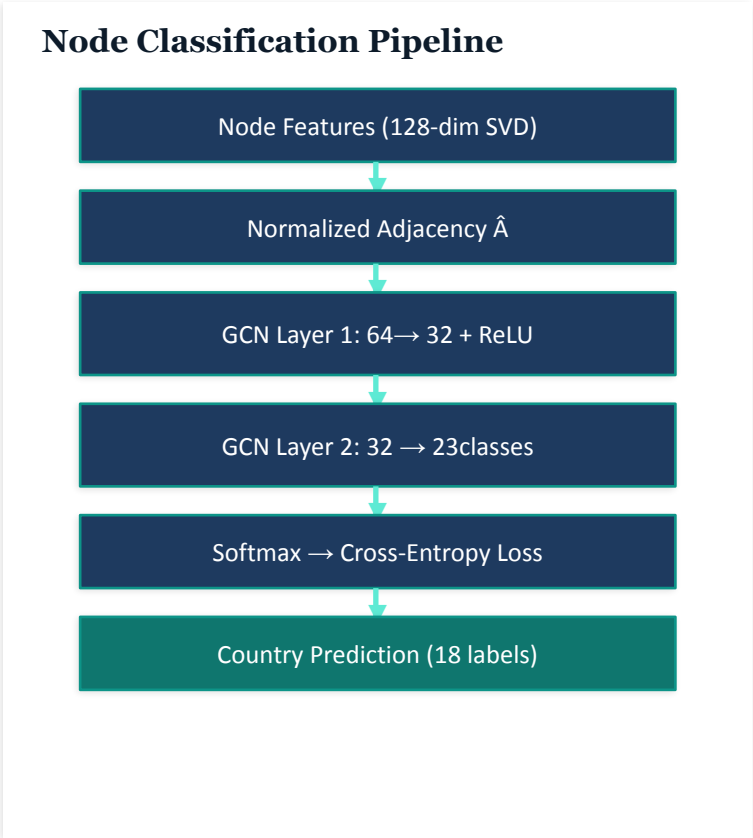
LastFM

Parameter	BFS-biased	DFS-biased	Role
`p`	1.0	2.0	Return parameter (controls probability of revisiting previous node).
`q`	2.0	0.5	In–out parameter (controls BFS vs DFS behavior).
`num_walks`	10	10	Number of walks per node.
`walk_length`	80	80	Length of each random walk.
`vector_size`	128	128	Embedding dimensionality.
`window`	10	10	Skip-gram context window size.
`epochs`	5	5	Word2Vec training epochs.
`n_clusters` (KMeans)	`n_classes`	`n_classes`	Number of clusters (equal to ground truth classes).

Cisco

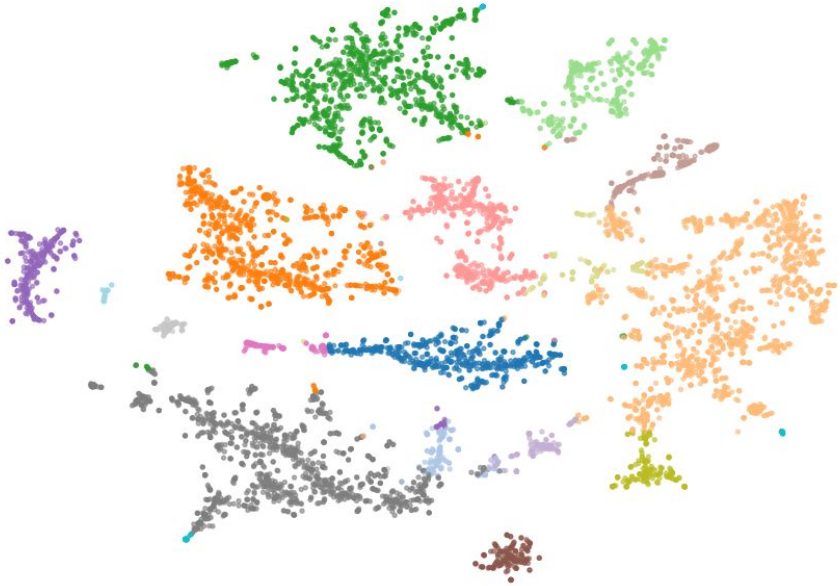
Parameter	BFS-biased	DFS-biased	Role
`p`	1.0	2.0	Return parameter (controls backtracking).
`q`	2.0	0.5	In–out parameter (controls BFS vs DFS behavior).
`num_walks`	20	20	Number of walks per node.
`walk_length`	40	40	Length of each random walk.
`vector_size`	64	64	Embedding dimensionality.
`window`	5	5	Skip-gram context window size.
`epochs`	10	10	Word2Vec training epochs.
`n_clusters` (KMeans)	23	23	Number of clusters (equal to ground truth groups).

GCN - Cisco pipeline configuration

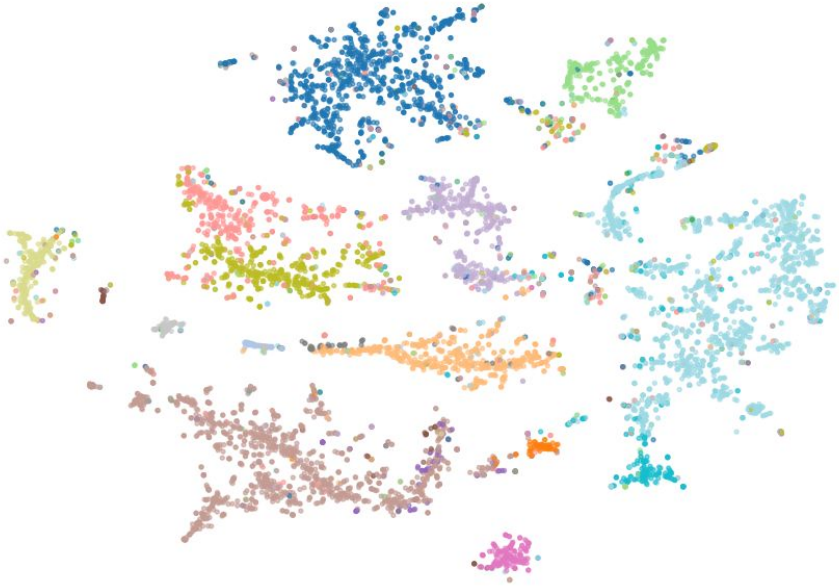


Spectral Embeddings Projected with t-SNE

Spectral Clustering (k=18)



Ground Truth (Country Labels)



```
# 60% train / 20% val / 20% test
perm = np.random.permutation(n_nodes)
train_mask[perm[:n_train]] = True
val_mask[perm[n_train:n_train+n_val]] = True
test_mask[perm[n_train+n_val:]] = True

 loss = F.cross_entropy(out[train_mask], y[train_mask])  # only train nodes
```

Node2Vec: Biased Random Walks → Shallow Embeddings

Pipeline

Graph G

↓

Biased Random Walks
p, q control BFS vs DFS

↓

Word2Vec (Skip-gram)

↓

Node Embeddings
+ K-Means ⟶ Communities

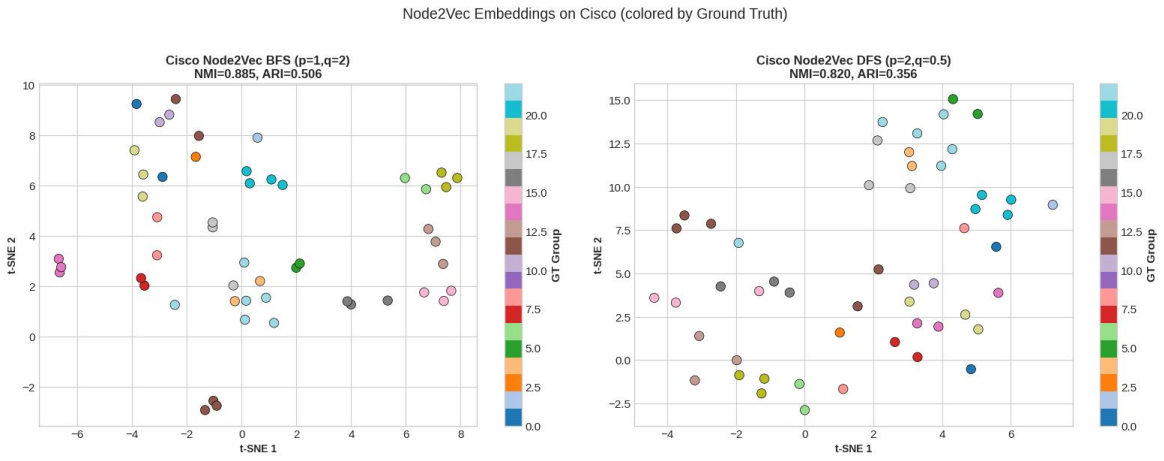
BFS (p=1, q=2): local structure
DFS (p=2, q=0.5): global homophily

t-SNE stands for **t**-distributed Stochastic Neighbor Embedding, a nonlinear dimensionality reduction method mainly used for visualization, not for downstream modeling.

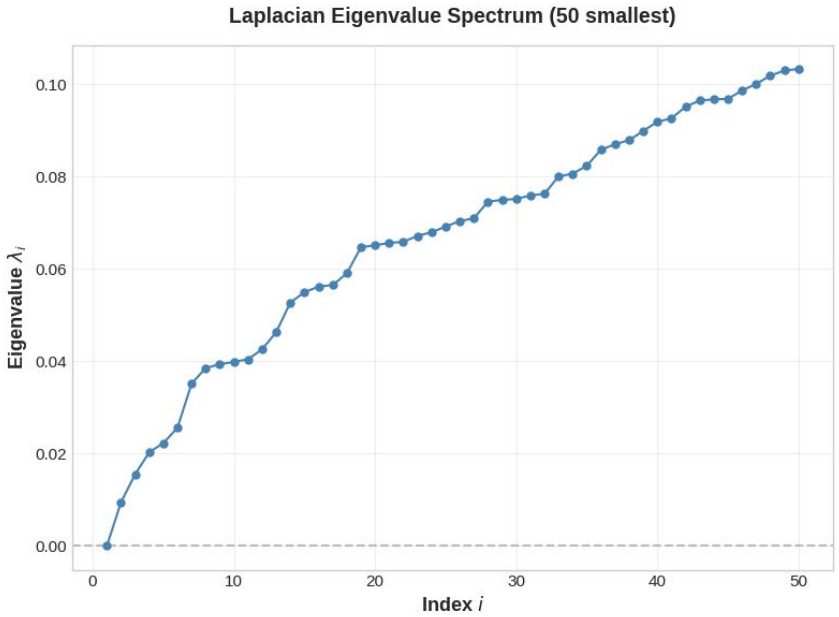
Cisco Results

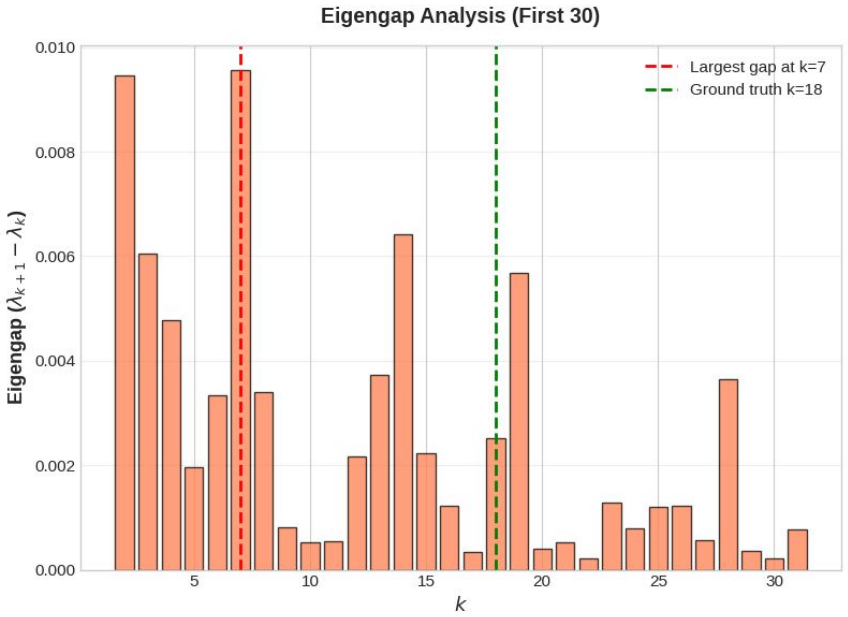
Config	Mod.	NMI	ARI
BFS (p=1, q=2)	0.0415	**0.8851**	**0.5056**
DFS (p=2, q=0.5)	**0.1828**	0.8196	0.4236

DFS > BFS: global structure captures more community signal. Neither beats Louvain on modularity or spectral on NMI. Cisco: poor (too dense for random walks).



LastFM






```
=== Spectral Clustering: Eigengap (k=7) ===
Clusters: 7, Sizes: [1770, 1732, 1477, 1142, 656, 621, 226]...
Modularity: 0.7641
NMI: 0.5966, ARI: 0.5584, AMI: 0.5952

=== Spectral Clustering: GT-matched (k=18) ===
Clusters: 18, Sizes: [1487, 1145, 1099, 980, 542, 514, 410, 306, 221, 170]...
Modularity: 0.7992
NMI: 0.6586, ARI: 0.6714, AMI: 0.6557
```

+ Code + Markdown

Cisco

```
…   Computing 40 smallest eigenvalues of Cisco Laplacian...
    Eigengap suggests k=2 clusters
    GT has 23 groups

    Spectral Clustering (k=2, Eigengap):
      Modularity: 0.0425
      NMI: 0.0793, ARI: 0.0021, AMI: 0.0170

    Spectral Clustering (k=23, GT-matched):
      Modularity: 0.2049
      NMI: 0.8632, ARI: 0.4961, AMI: 0.5539
```